Date: October 10, 2026

Artificial Intelligence - Lab

Lesson 6 - Conversational State Machines and Workflows

Lab Activity 6.1: Stateful Restaurant Order Agent & Dynamic Tool Calling Engine

Business Scenario

Perpetual Bistro, a high-volume campus dining facility, is prototyping an automated table-side digital ordering system. Customers place complex orders, request modifications (e.g., "Extra cheese, no onions, make it vegetarian"), ask about menu ingredients, and add/remove items dynamically before confirming their order.

The bistro needs a local, zero-cloud conversational engine that maintains active table session memory, checks real-time menu availability from an SQLite database, dynamically updates the customer's tab, and prevents hallucinated menu items.

Lab Description

Working in pairs (**Driver & Navigator**), you will build a Python CLI application powered by Ollama (qwen2.5) and SQLite. The application acts as a stateful order intake agent that maintains session state using an FSM pattern.

Step-by-Step Implementation:

1. **Database Setup (restaurant.db):** Initialize SQLite tables for menu_items (id, name, category, price, stock_qty, allergens), table_sessions (session_id, table_no, status), and order_line_items (session_id, item_id, quantity, special_instructions).
2. **Tool Definition & Function Binding:** Define Python functions that the LLM can invoke:
    - get_menu_by_category(category)
    - check_item_allergen(item_id, allergen)
    - add_item_to_order(session_id, item_name, qty, instructions)
    - view_current_order(session_id)
3. **Conversational FSM Loop:** Construct a Python state loop containing 5 distinct states:
    - STATE_GREETING: Welcome customer, capture table number.
    - STATE_TAKING_ORDER: Handle queries, recommend items, and bind tool calls to add/modify items.
    - STATE_REVIEW: Present current itemized tab with calculated totals for customer verification.
    - STATE_CONFIRMED: Commit transactional changes to restaurant.db, update stock levels, and output order receipt.

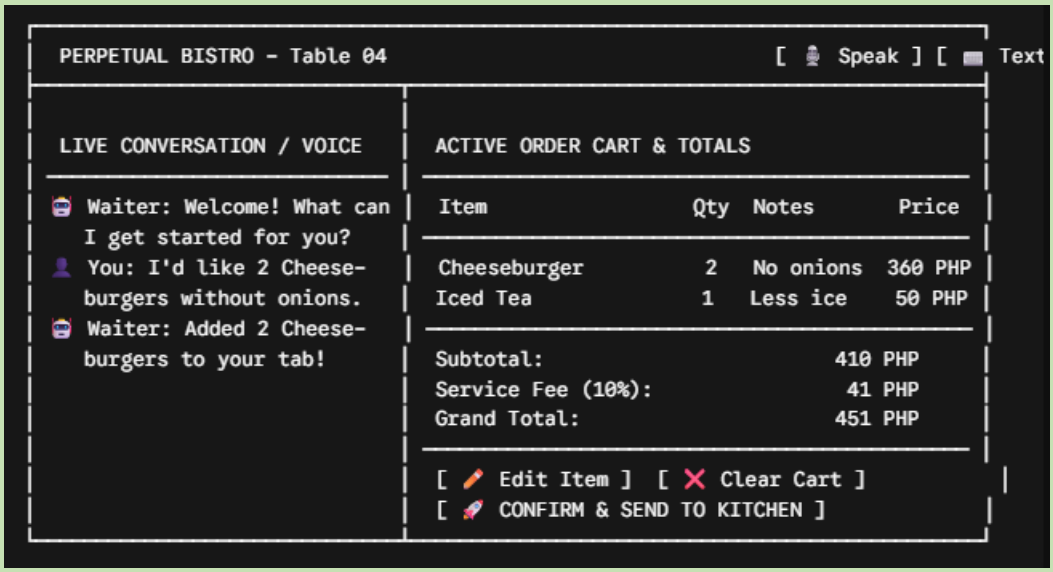
4. **Tandem Collaboration Requirement:** Pairs must rotate roles (Driver codes while Navigator reviews logic/prompts) halfway through the lab.

Expected Outcome:

An interactive, stateful Python CLI application (virtual_waiter_prototype.py) where a user can simulate a multi-turn ordering conversation. The agent remembers previous turns, calls SQLite tools automatically, updates the order tab accurately, and successfully handles order modifications.

Option 1: Web-Based Dashboard — Highly Recommended

A lightweight web dashboard running locally via Python (localhost:5000 or localhost:8000) provides the most realistic restaurant table-top tablet ordering experience.



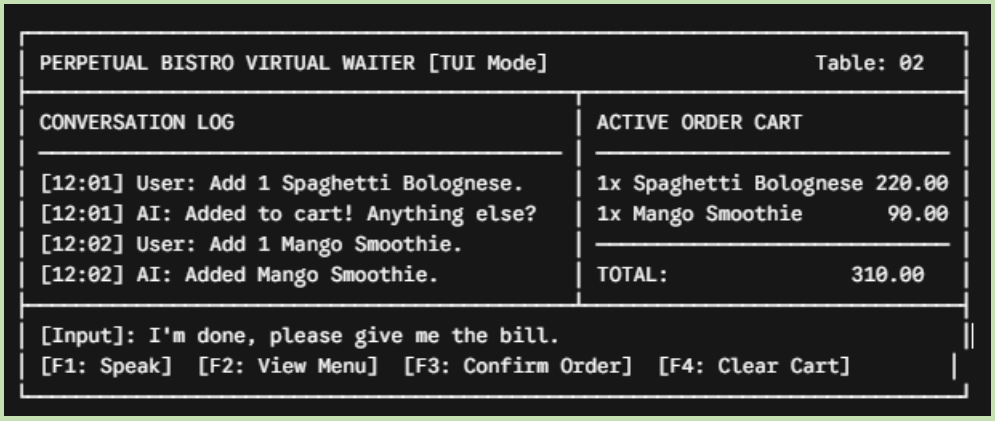
Key Components of the Web UI Layout:

1. **Left Panel — Conversation & Voice Stream:**
    - Visual transcript of student-AI dialogue.
    - **Voice Toggle Button ( ):** Indicates STT/TTS status (Active/Listening/Speaking).
    - **Manual Text Input:** Backup input bar for testing without microphone audio.
2. **Right Panel — Real-Time Order Cart (Synced to SQLite):**
    - **Live Dynamic Table:** Updates instantly via asynchronous polling or WebSockets whenever Qwen triggers a tool call (add_item_to_order, remove_item).
    - **Customization Badges:** Displays special instructions (e.g., No onions, Extra cheese) attached to specific line items.
    - **Interactive Controls:** Manual quantity adjusters (+ / -) allowing human-in-the-loop overrides if the user prefers clicking over talking.
3. **Bottom Bar — Checkout & FSM Control:**

- Prominent **"CONFIRM ORDER"** button that triggers the FSM state transition from REVIEW to CONFIRMED.
- Digital receipt generator modal upon checkout completion.

Option 2: Terminal-Based Rich TUI (Textual / Rich Library)

For a hyper-lightweight setup that runs 100% inside the terminal while still providing a structured, visual interface, students can use Python's **Textual** library.



List of Deliverables:

1. init_restaurant_db.sql & db_tools.py: SQLite schema initialization script and Python database tool execution functions.
2. fsm_agent.py: Finite State Machine controller managing conversation states and session memory.
3. virtual_waiter_prototype.py: Executable CLI application running the interactive agent loop powered by Ollama and Qwen.
4. pair_collaboration_log.md: A short tandem programming log detailing role rotations, driver-navigator task breakdowns, and key design decisions made by the pair.
5. sample_conversation_log.json: A saved JSON log capturing a multi-turn conversation showing tool calls, state transitions, and database state updates.

Assessment Rubric (Pair Evaluation):

Criteria	Exemplary (4)	Proficient (3)	Developing (2)	Unacceptable (1)
State Machine & Session Memory	Conversation flows seamlessly through all FSM states. Preserves multi-turn context without	Handles multi-turn memory well; minor edge cases cause state transition	Frequently loses track of context or resets session state mid-conversation.	Application is stateless; treats every user input as an isolated query.

Criteria	Exemplary (4)	Proficient (3)	Developing (2)	Unacceptable (1)
	dropping items or resetting order state.	glitches, but recovery is possible.		
Tool Calling & Database Binding	Qwen reliably triggers correct Python/SQLite tools with accurate arguments; updates inventory and tab accurately.	Tool calls execute successfully, but requires minor prompt hints or handles edge cases poorly.	Tool calling frequently fails or passes wrong arguments to SQLite functions.	No tool binding implemented; data is hardcoded or hallucinated by the LLM.
Robustness & Edge Handling	Politely handles out-of-stock items, invalid requests, and order edits/cancellations without crashing or hallucinating.	Handles basic order changes well, but fails on complex edge cases (e.g., removing a non-existent item).	System crashes or hallucinates menu items when users provide unexpected inputs.	System crashes easily or responds with unstructured error traces.
Tandem Collaboration & Code Quality	Code is highly modular, well-commented, and pair log demonstrates equal, active participation in Driver/Navigator roles.	Code is clean and functional; pair log indicates work distribution with minor imbalances.	Poor code modularity; pair log indicates one student performed most of the work.	Unorganized code; no evidence of tandem collaboration or pair logging.

Due date: October 17, 2026